Python 102 - Week 2 Master Reference: Linear Data Structures (Stacks & Queues)

Welcome to your complete consolidated master notebook for **Week 2**. This reference document merges, organizes, and explains all concepts, code implementations, mental models, memory mechanics, and review exercises from the 11 course PDF sources.

It is structured to be completely self-contained and formatted in markdown for direct import into your master class notebook.

Table of Contents

1. Abstract Linear Data Structures & LIFO vs. FIFO
2. The Stack Data Structure (LIFO Logic)
3. The Queue Data Structure (FIFO Logic)
4. Python Memory Mechanics: References, Mutability, and Function Scopes
5. High-Performance Queues with `collections.deque`
6. Advanced Integration: Hybrid Stack-Queue Systems
7. Step-by-Step Code Tracing Challenges & Diagnostics

---

1. Abstract Linear Data Structures & LIFO vs. FIFO

Linear data structures organize data sequentially. The two fundamental abstract linear data structures are **Stacks** and **Queues**. While both represent collections of elements, they process and access data in opposite "temporal directions" or rules.

Side-by-Side Comparison Matrix

Specification	Stack	Queue
Ordering Rule	**LIFO**: Last-In, First-Out	**FIFO**: First-In, First-Out
Access Point	Single end only (the **Top**)	Two ends (add at **Rear**, remove from **Front**)
Real-World Analogy	Pile of books on a shelf, stack of buffet plates	Waiting line for a bus, ice-cream truck queue
Primary Operations	Push (add to top), Pop (remove from top)	Enqueue (add to back), Dequeue (remove from front)

Use-Case Fit	Browser back-button, text-editor undo histories	Printer queues, game server matchmaking lobby

2. The Stack Data Structure (LIFO Logic)

A **Stack** is an ordered collection of items where all insertions (pushes) and removals (pops) happen at a single end called the **Top**. It operates strictly on the **LIFO** (Last-In, First-Out) principle.

Real-World Mental Models

- **Stack of Dinner Plates in a Cupboard**: Plates are stacked on top of each other. You only add a plate to the top of the pile, and it is easiest and most logical to remove plates from the top first. To reach a plate at the bottom, you have to work your way down.
- **Undo History in a Text Editor**: As you type words or edit text (e.g., "Rep" $\rightarrow$ "Repl" $\rightarrow$ "Reply"), each state is pushed onto an undo history stack. When you hit `Ctrl+Z` (Undo), the most recent change is popped off first.
- **Browser Back-Button**: When browsing, visiting a new page places it on top of your history. Pressing "Back" returns you to the most recent page first.

Python List-Based Stack Implementation

In Python, we conceptually map a standard **list** to a stack where the **right end (index `-1`) of the list is the top of the stack**, and the **left end (index `0`) is the bottom** (oldest item).

```python
# Starting an empty stack
stack = []

# 1. PUSH OPERATION (Using .append)
# append() adds an element to the rightmost end (top) of the list
stack.append("A")  # Stack state: ["A"] (A is bottom & top)
stack.append("B")  # Stack state: ["A", "B"] (B is now on top)
stack.append("C")  # Stack state: ["A", "B", "C"] (C is now on top)

# 2. PEEK OPERATION (Look at the top without removing it)
# We access index -1 to inspect the top of the stack safely
if len(stack) > 0:
    top_item = stack[-1]
    print("Top item (Peek):", top_item)  # Prints: 'C'

# 3. POP OPERATION (Remove and retrieve the top item)
# pop() modifies the list by removing the rightmost item and return
popped_item = stack.pop()
print("Popped Item:", popped_item)     # Prints: 'C'
print("Stack After Pop:", stack)       # Prints: ['A', 'B']
```

Memory Safety & Underflow Guards

Attempting to peek (access `stack[-1]`) or pop (call `stack.pop()`) from an empty list `[]` causes Python to crash with an `IndexError`. To protect against this, you must always implement an **underflow guard**.

Classical Length Check Guard

```python
if len(stack) == 0:
    print("Stack is empty! Cannot pop or peek.")
else:
    top_item = stack.pop()
    print("Safely popped:", top_item)
```

Pythonic Implicit Boolean Guard

In Python, an empty list evaluates to `False` in a boolean context, and a non-empty list evaluates to `True`.

```python
if stack:
    top_item = stack.pop()
    print("Safely popped:", top_item)
else:
    print("Stack is empty!")
```

Case Study: Multi-Stage Drawing App History (Double-Undo)

If you need to implement a "double-undo" (reverting the last two actions at once), you must ensure that there are at least two items on the stack before invoking consecutive pops.

```
history = ["stroke 1", "stroke 2", "stroke 3", "stroke 4"]

# Double-Undo with Safe Underflow Guard
if len(history) >= 2:
    undone_1 = history.pop()  # Removes "stroke 4"
    undone_2 = history.pop()  # Removes "stroke 3"
    print(f"Undid actions: '{undone_1}' and '{undone_2}'")
    print("Remaining History:", history)  # Prints: ['stroke 1', 'st
else:
    print("Not enough actions in history to perform a double undo!"
```

---

3. The Queue Data Structure (FIFO Logic)

A **Queue** is an ordered collection where items are added at one end (the **Rear/Back**) and removed from the opposite end (the **Front**). It operates strictly on the **FIFO** (First-In, First-Out) principle.

Real-World Mental Models

- **Coffee Shop Line**: The first customer to join the line stands at the counter (front) and is served first. New customers join at the back of the line. No cutting or skipping is allowed.
- **Office Print Queue**: Multiple documents are sent to a printer. The printer processes the documents in exact arrival order.

Python List-Based Queue Implementation

When modeling a queue with a standard Python list, we map:

- **Front of the queue** $\rightarrow$ **Index `0`**
- **Rear of the queue** $\rightarrow$ **Last index (`len(queue) - 1`)**

```
# Start with an empty queue
queue = []

# 1. ENQUEUE OPERATION (Add to rear via .append)
queue.append("Alice")  # Queue: ["Alice"]
queue.append("Ben")    # Queue: ["Alice", "Ben"]
queue.append("Cara")   # Queue: ["Alice", "Ben", "Cara"]
# Alice is at index 0 (Front), Cara is at index 2 (Rear)

# 2. DEQUEUE OPERATION (Remove from front via .pop(0))
# pop(0) removes the item at index 0 and shifts all remaining eleme
if len(queue) > 0:
    served_customer = queue.pop(0)
    print("Served:", served_customer)  # Prints: 'Alice'
    print("Queue Now:", queue)         # Prints: ['Ben', 'Cara']
```

Underflow Guard for List-Based Queue

```
def dequeue(queue_list):
    if len(queue_list) > 0:  # or simply: 'if queue_list:'
        return queue_list.pop(0)
    else:
        return "Queue is empty"
```

---

4. Python Memory Mechanics: References, Mutability, and Function Scopes

An important part of working with data structures in Python is understanding how arguments are passed into functions. Python uses **pass-by-object-reference**.

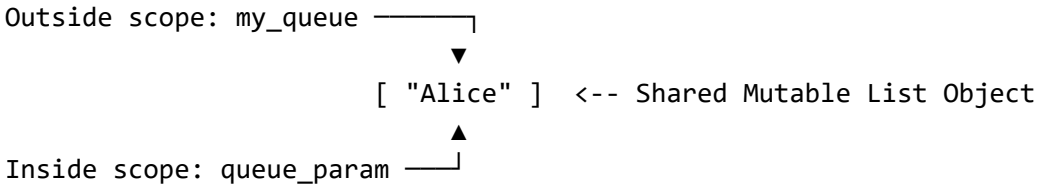
Mutability in Place

Lists are **mutable objects**, meaning they can be modified in place. When you pass a list to a function, the parameter inside the function points to the **exact same memory address** as the list outside the function.

```
def enqueue(queue_param, item):
    queue_param.append(item)  # Modifying the shared list object in

# Main Program Scope
my_queue = []
enqueue(my_queue, "Alice")
print(my_queue)  # Prints: ['Alice']
```

- **Why no `return` statement is needed**: Since `my_queue` and `queue_param` are two names referring to the same object in memory, modifications made inside enqueue are instantly visible outside.



Reference Sharing Hazards (A Cautionary Note)

If you create a collection of mutable objects (e.g., a list of dictionaries or a list of list queues) and do not perform a **deep copy**, copy operations will only copy the *references* to the objects, not the objects themselves. Modifying a dictionary inside one list would unexpectedly mutate it across all other list references!

---

5. High-Performance Queues with `collections.deque`

While standard lists work perfectly for stacks, using `list.pop(0)` for a queue dequeue is computationally expensive. Because lists are stored in contiguous memory, popping from index `0` forces Python to shift **every single remaining element** one position to the left. This makes list-based dequeue an $O(n)$ operation.

To resolve this bottleneck, Python provides a high-performance double-ended queue class in its standard library: `collections.deque`.

Importing Built-in Modules

Python standard modules are loaded using two primary import styles:

1. **Whole-Module Import**:

```
import math
result = math.sqrt(16)  # Requires referencing the module name
```

2. **Selective Tool Import**:

```
from math import sqrt
result = sqrt(16)       # Imports tool directly into local sco
```

Working with `collections.deque`

To use a double-ended queue, import it from the built-in `collections` module.

```
from collections import deque

# Creating an empty deque
queue = deque()

# ENQUEUE: We use append() to add to the rear (right end)
queue.append("customer1")
queue.append("customer2")
queue.append("customer3")
print("Initial deque:", queue)  # Prints: deque(['customer1', 'custo

# DEQUEUE: We use popleft() to remove from the front (left end)
# deque.popleft() is optimized to run in O(1) constant time!
served_1 = queue.popleft()
print("Served:", served_1)      # Prints: 'customer1'
print("Remaining Deque:", queue) # Prints: deque(['customer2', 'cust
```

Queues in Larger Systems: Flow of Work Control

In advanced algorithms, a queue's primary responsibility is often not to sort or reorganize data itself, but rather to **control the flow of work**.

- Items are queued as they become "ready" to be processed.
- The system repeatedly dequeues the front item, processes it (which might involve placing it into a separate sorted structure), and moves on to the next.
- This separates work execution scheduling (FIFO queue) from sorting/priority rules.

---

6. Advanced Integration: Hybrid Stack-Queue Systems

In real-world applications, different linear structures are combined to model complex processes. An excellent example is a **FIFO queue of characters, where each character carries a LIFO stack (backpack) of inventory items**.

Case Study: Dungeon Adventure Simulation

Imagine a line of adventurers waiting at a dungeon gate.

1. The line at the gate is a **Queue (FIFO)**: the first to arrive enters first.
2. Each adventurer carries a backpack modeled as a **Stack (LIFO)**: they throw gear on top, and must use their most recently added (top) item first.

```python
from collections import deque

# Setup the adventure queue (A deque of lists, where index 1 is a l
adventure_queue = deque([
    ["Alice", ["sword", "potion", "map"]],      # Alice's backpack
    ["Bob", ["shield", "dagger"]],              # Bob's backpack to
    ["John", ["magical hat", "spell book"]],    # John's backpack t
])

print("--- Start of Dungeon Run ---")
print("Adventure queue at gate:", adventure_queue)

# Step 1: Alice enters the dungeon (Dequeued from front of line)
current_adventurer = adventure_queue.popleft()  # Deque operation (
adventurer_name = current_adventurer[0]
backpack_stack = current_adventurer[1]

# Step 2: She grabs the top item from her backpack (Popped from top
used_item = backpack_stack.pop()                # Stack operation (

print(f"\n[Action 1] {adventurer_name} entered the dungeon!")
print(f" > {adventurer_name} used their top item: '{used_item}'")
print(f" > {adventurer_name}'s remaining backpack items: {backpack_
print("Remaining queue at gate:", adventure_queue)

# Step 3: Bob enters the dungeon next
current_adventurer = adventure_queue.popleft()  # Deque operation (
adventurer_name = current_adventurer[0]
backpack_stack = current_adventurer[1]

# Step 4: Bob uses his top backpack item
used_item = backpack_stack.pop()                # Stack operation (

print(f"\n[Action 2] {adventurer_name} entered the dungeon!")
print(f" > {adventurer_name} used their top item: '{used_item}'")
print(f" > {adventurer_name}'s remaining backpack items: {backpack_
print("Final queue at gate:", adventure_queue)
```

Detailed Trace Walkthrough

1. **Initial Queue State**:
    - Front: Alice (carrying `["sword", "potion", "map"]`)
    - Middle: Bob (carrying `["shield", "dagger"]`)
    - Rear: John (carrying `["magical hat", "spell book"]`)
2. **First `popleft()` (FIFO)**:
    - Removes **Alice** from the front. Queue becomes: `[Bob, John]`
3. **First `pop()` (LIFO)**:

- Removes **"map"** (rightmost element) from Alice's backpack. Alice is left with `["sword", "potion"]`.

4. **Second `popleft()` (FIFO)**:
   - Removes **Bob** from the front. Queue becomes: `[John]`
5. **Second `pop()` (LIFO)**:
   - Removes **"dagger"** (rightmost element) from Bob's backpack. Bob is left with `["shield"]`.

What if the rules change?

If the story rules shifted so that "each adventurer must use their oldest item first," a backpack managed as a LIFO stack would no longer make sense. Instead, each backpack would need to be treated as a **Queue (FIFO)**, where the first item packed is the first item used.

---

7. Step-by-Step Code Tracing Challenges & Diagnostics

Lock in your understanding of execution order by tracing these challenges.

Challenge A: Browser Back History Trace

```
stack = ["home", "news", "article"]  # top is on the right ("article

stack.pop()             # Removes "article". Stack: ["home", "news"
stack.append("profile") # Pushes "profile". Stack: ["home", "news",
stack.append("settings")# Pushes "settings". Stack: ["home", "news",
top_page = stack.pop()  # Pops "settings". Stack: ["home", "news",

print(top_page)         # Output: "settings"
print(stack)            # Output: ["home", "news", "profile"]
```

Challenge B: Stack of Plates Value Trace

```
stack = [1, 2, 3]  # top is 3

stack.append(4)  # Stack: [1, 2, 3, 4]
x = stack.pop()  # Pops 4. x = 4. Stack: [1, 2, 3]
stack.append(5)  # Stack: [1, 2, 3, 5]
stack.append(6)  # Stack: [1, 2, 3, 5, 6]
y = stack.pop()  # Pops 6. y = 6. Stack: [1, 2, 3, 5]

print(f"x={x},y={y}")  # Output: x=4,y=6
print(stack)           # Output: [1, 2, 3, 5]
```

Challenge C: Mixed Stack Pushes and Pops

```
stack = [10]

stack.append(20)  # Stack: [10, 20]
stack.append(30)  # Stack: [10, 20, 30]
a = stack.pop()   # Pops 30. a = 30. Stack: [10, 20]
stack.append(40)  # Stack: [10, 20, 40]
b = stack.pop()   # Pops 40. b = 40. Stack: [10, 20]
c = stack.pop()   # Pops 20. c = 20. Stack: [10]

print(f"a={a}, b={b}, c={c}")  # Output: a=30, b=40, c=20
print(stack)                   # Output: [10]
```

Challenge D: List-Based Queue Trace

```
queue = []
queue.append(1)        # Queue: [1]
queue.append(2)        # Queue: [1, 2]
queue.append(3)        # Queue: [1, 2, 3]
first = queue.pop(0)   # Removes from front. first = 1. Queue: [2, 3
queue.append(4)        # Queue: [2, 3, 4]
second = queue.pop(0)  # Removes from front. second = 2. Queue: [3,

print(f"first={first}, second={second}")  # Output: first=1, second=
print(queue)                              # Output: [3, 4]
```

End of Reference Document. Ready to be imported directly into your Master Class Notebook!